# Python 3.12 AsyncIO & High-Performance Coroutines • STUDENT EXAM COPY

Course: Python 3.12.8 • College: VIT • Student Name: _____

### Q1. In Python 3.12, how does asyncio manage concurrent I/O operations without spawning OS threads?

A) Using a single-threaded cooperative event loop that yields control at 'await' expressions

B) By running each coroutine on a separate physical CPU hardware core simultaneously

C) By disabling the Global Interpreter Lock (GIL) automatically for all functions

D) By converting all asynchronous Python functions into synchronous C-extensions at runtime

### Q2. Which Python interpreter flag treats PendingDeprecationWarning and FutureWarning as fatal runtime exceptions?

A) `-W error::DeprecationWarning -W error::FutureWarning`

B) --ignore-deprecation-errors

C) -O2 --disable-warnings

D) -B --no-exceptions

### Q3. What is the exact stdout produced by running this verified Python snippet?

```python
nums = [1, 2, 3, 4]
res = [x * 2 for x in nums if x % 2 == 0]
print(res)
```

A) [2, 4, 6, 8]

B) [4, 8]

C) [2, 6]

D) [4, 6, 8]

### Q4. What is the exact stdout produced by running this verified Python snippet?

```python
data = {'a': 10, 'b': 20}
val = data.get('c', 30) + data['a']
print(val)
```

A) 10

B) 40

C) 30

D) KeyError: 'c'

### Q5. In the hands-on lab exercise on Slide 10, what critical edge case must the implementation handle to pass all test assertions?

A) Empty or null collections must return a neutral default value (e.g. 0) without throwing exceptions

B) The function must always assume at least 10 positive integers are present

C) The method must catch and swallow all exceptions silently without returning any value

D) The function must sort the input collection in-place regardless of immutability

### Q6. During automated verification of the exercise on Slide 10, why does the naive solution fail the assertion check?

A) It computes an unchecked aggregate (e.g. total length or only first elements) without validating required element constraints

B) It fails to compile because modern compilers reject the 'return' keyword inside loops

C) It exhausts all system RAM by spawning 1,000,000 OS background processes

D) It triggers a hardware floating-point division-by-zero error on string literals

### Q7. What is the exact stdout produced by running this verified Python snippet?

```
vals = [10, 20, 30]
ans = sum(v // 10 for v in vals)
print(ans)
```

A) 60

B) 6

C) 3

D) 6.0

### Q8. As emphasized in the summary on Slide 13, why must automated verification normalize memory addresses and timestamps?

A) To ensure deterministic test assertions by replacing non-deterministic pointer addresses (e.g. 0x7ffe...) and timestamps with stable placeholders

B) To compress the source code so it can fit on older floppy disks

C) Because the operating system forbids printing hexadecimal numbers to stdout

D) To convert all terminal output into uppercase text automatically